

Solidity Audit Platform

Статус продукта: beta v0.1 / pre-release

Область применения: автоматизированный аудит безопасности смарт-контрактов Solidity и проектов для EVM-совместимых сетей

Дата: 2026-06-03

Данный документ описывает техническую, продуктовую и организационную логику проекта Solidity Audit Platform. Whitepaper не является юридической, финансовой или инвестиционной рекомендацией. Его задача — объяснить, какую проблему решает проект, как устроена система анализа, какие классы уязвимостей она помогает обнаруживать и какую роль такая платформа может занимать в процессе разработки смарт-контрактов.

1. Название и миссия проекта

Solidity Audit Platform — это платформа для автоматизированного анализа безопасности смарт-контрактов, написанных на Solidity, а также проектов, предназначенных для запуска в EVM-совместимых сетях.

Миссия проекта состоит в том, чтобы сделать первичный аудит смарт-контрактов более доступным, воспроизводимым и понятным для разработчиков. На практике команды часто используют отдельные инструменты: Slither для статического анализа, Mythril для символьного исполнения, Foundry для сборки и тестов, Echidna для проверки инвариантов. Однако каждый из этих инструментов требует настройки, отдельного запуска и ручной интерпретации результата. Solidity Audit Platform объединяет эти проверки в единый процесс, сохраняет технические логи и приводит результаты к общему формату.

Платформа не пытается заменить профессиональный ручной аудит. Ее роль другая: дать команде быстрый предварительный слой проверки до публикации контракта в сети. Такой слой помогает обнаружить очевидные ошибки, увидеть рискованные участки кода, подготовиться к полноценному аудиту и сократить количество дефектов еще на этапе разработки.

2. Проблема, которую решает проект

Смарт-контракты отличаются от обычного прикладного программного обеспечения тем, что после публикации в блокчейне их часто невозможно просто исправить без заранее предусмотренного механизма обновления. Если ошибка есть в backend-

сервисе, ее можно закрыть патчем, перезапустить сервер и восстановить данные из резервной копии. В случае смарт-контракта уязвимость может сразу привести к потере активов, блокировке средств, нарушению логики управления или атаке на экономическую модель протокола.

Типичный пример — reentrancy. Контракт хранит балансы пользователей и позволяет им выводить средства. Если функция `withdraw` сначала отправляет эфир внешнему адресу, а только потом уменьшает баланс пользователя, злоумышленник может создать контракт-получатель, который повторно вызовет `withdraw` во время первого вывода. В результате логика контракта будет выполняться в промежуточном состоянии, где баланс еще не уменьшен. Такой сценарий исторически является одним из самых известных классов атак на EVM-контракты.

Другой распространенный пример — использование `tx.origin` для проверки владельца. Разработчик может написать условие вида `require(tx.origin == owner)`, предполагая, что так он проверяет настоящего пользователя. Но если владелец взаимодействует с вредоносным контрактом, тот может вызвать целевой контракт дальше по цепочке, и `tx.origin` все равно останется адресом владельца. В результате злоумышленник получает возможность обходить авторизацию через фишинговый сценарий. Безопаснее использовать `msg.sender`, потому что он отражает непосредственного вызывающего.

Опасность представляют и слабые источники случайности. Например, если контракт вычисляет результат лотереи через `block.timestamp`, `block.prevrandao` или другие предсказуемые параметры блока, участник или валидатор может получить преимущество. В приложениях с финансовым стимулом такая псевдослучайность обычно недостаточна. Для критичных сценариев требуется специально спроектированная схема случайности, например с использованием внешнего проверяемого источника.

Еще один класс ошибок связан с учетом балансов. Контракт может хранить внутренний баланс пользователя в `mapping`, но фактический баланс самого контракта в эфире или токенах может перестать соответствовать сумме внутренних записей. Такая рассинхронизация часто возникает при прямых переводах на контракт, неверной обработке `receive` / `fallback`, ошибках округления, неучтенных комиссиях, неправильной работе с токенами или нарушении инвариантов. В простых случаях это можно обнаружить property-based fuzzing: формулируется свойство, например “баланс контракта должен покрывать внутренние балансы пользователей”, после чего Echidna пытается найти последовательность вызовов, нарушающую это свойство.

Ошибки доступа также остаются одной из главных причин инцидентов. Если функция выпуска токенов, обновления адреса оракула, смены реализации роху-контракта или вывода средств не защищена корректной проверкой прав, атакующий может получить управление над критической логикой. Даже если в коде есть модификатор `onlyOwner`, важно проверить, кто задает владельца, можно ли его заменить, не доступна ли

функция инициализации повторно и не нарушается ли модель доступа при обновлении контракта.

Проблема состоит не только в наличии уязвимостей, но и в сложности их систематического поиска. Отдельные инструменты дают разные форматы вывода, используют разные термины, иногда возвращают ложные срабатывания, а иногда падают из-за настроек компилятора, зависимостей или структуры проекта. Solidity Audit Platform решает эту проблему как интеграционный слой: она принимает код, запускает несколько анализаторов, сохраняет логи, нормализует findings и показывает пользователю общую картину.

3. Техническое решение

Solidity Audit Platform является off-chain системой анализа. Она не является отдельным блокчейном, не вводит собственный механизм консенсуса и не выполняет транзакции в сети. Пользователь загружает `.sol` файл или `.zip` архив с проектом, после чего backend создает запись проекта, сохраняет исходные файлы в изолированной директории и запускает анализ.

Основу системы составляет FastAPI backend. Он отвечает за загрузку проектов, запуск анализа, получение статуса, выдачу findings, просмотр логов и работу WebSocket-обновлений. Долгие задачи выполняются асинхронно через Celery worker. Redis используется как брокер задач, PostgreSQL — как основное хранилище данных. Для тяжелых анализаторов применяется Docker-слой: Slither, Mythril, Foundry и Echidna запускаются в отдельных контейнерах с ограничениями по памяти, процессам и сети.

Такой подход важен по двум причинам. Во-первых, анализаторы имеют сложные зависимости и могут конфликтовать между собой. Mythril, Slither, Foundry и Echidna удобнее изолировать в отдельных образах, чем пытаться установить все в одно окружение. Во-вторых, пользовательский код нельзя считать доверенным. Даже если платформа анализирует только исходники, безопаснее запускать инструменты в контейнерах без доступа к сети и с read-only монтированием проекта.

Внутри системы каждая проверка рассматривается как отдельный инструмент. Basic Scanner и пользовательские CFG/DFG-проверки выполняются как Python-код backend-a. DockerRunner запускает внешние контейнеры и возвращает stdout, stderr, код завершения и признак тайм-аута. Результаты инструментов преобразуются в единый формат finding: severity, rule, tool, message, location и дополнительные служебные поля. Отдельно сохраняются технические логи, чтобы пользователь мог понять, почему инструмент нашел проблему или почему он завершился ошибкой.

Pipeline не обязан останавливаться при сбое одного инструмента. Например, если Mythril не смог проанализировать один файл из-за ограничения времени, это не должно отменять результаты Slither, Foundry или CFG/DFG. Такой режим называется

частичным успехом. Он особенно важен для практического аудита, потому что даже неполный отчет лучше, чем полный отказ всей проверки.

4. Анализаторы и типы обнаруживаемых проблем

Basic Scanner выполняет быстрые эвристические проверки. Он не строит полноценное дерево разбора Solidity-кода, но способен находить очевидные рискованные конструкции. Например, он обнаруживает `tx.origin`, `selfdestruct`, `delegatecall`, низкоуровневые вызовы `.call`, использование `block.timestamp`, отсутствие `pragma solidity` или SPDX-идентификатора. Такой анализатор полезен как первый фильтр: он быстро показывает потенциально опасные места, но его результаты должны подтверждаться более глубокими инструментами.

Slither выполняет статический анализ. Он может обнаруживать reentrancy, низкоуровневые вызовы, слабую псевдослучайность, опасные зависимости от версии компилятора, подозрительные шаблоны наследования, ошибки видимости и другие проблемы. Например, для функции вывода средств Slither может показать, что внешний вызов происходит до изменения состояния, а значит существует риск повторного входа. В отличие от простого регулярного поиска, Slither учитывает структуру контракта и связи между функциями.

Mythril использует символьное исполнение. Он пытается исследовать возможные пути выполнения контракта и найти состояния, в которых возникает уязвимость. Например, Mythril может обнаружить, что после внешнего вызова выполняется чтение или запись постоянного состояния. Это усиливает вывод о reentrancy-риске, потому что проблема подтверждается не только синтаксическим шаблоном, но и анализом пути выполнения. При этом Mythril может быть тяжелым по времени и памяти, поэтому для него особенно важны тайм-ауты и сохранение полного технического вывода.

Foundry используется для сборки и запуска тестов. Если проект компилируется, но один из тестов падает, это не означает, что платформа сломалась. Это означает, что тестовый сценарий обнаружил нежелательное поведение или демонстрационную атаку. Например, тест `testReentrancyAttack` может запускать атакующий контракт, который повторно вызывает `withdraw`. Если цепочка завершается `transfer failed` или `panic` из-за `underflow/overflow`, такой результат нужно классифицировать как `failing test`, а не как ошибку инфраструктуры. Для одиночного `.sol` файла Foundry может просто подтвердить успешную компиляцию и сообщить, что тестов нет.

Echidna проверяет свойства и инварианты. В отличие от обычного теста, где разработчик заранее задает конкретную последовательность вызовов, fuzzing генерирует множество последовательностей автоматически. Например, если в контракте есть свойство `echidna_contract_balance_covers_internal_balances`, Echidna может найти последовательность `deposit`, `fallback`, `withdraw`, после

которой внутренний учет перестает соответствовать фактическому балансу. В этом случае код завершения `1` является не поломкой инструмента, а успешным обнаружением нарушения свойства. Платформа должна показывать такой результат как `ECHIDNA_PROPERTY_FAILED`.

CFG и DFG — это пользовательские анализаторы графа управления и потоков данных. CFG помогает увидеть функции, условия и внешние вызовы. DFG фиксирует чтение и запись переменных состояния. Сами по себе такие факты не всегда являются уязвимостью, но они дают основу для корреляционного анализа. Например, если CFG показывает внешний вызов в функции `withdraw`, а DFG показывает запись `balances[msg.sender] -= amount` после этого вызова, система может сформировать finding `POSSIBLE_REENTRANCY_BY_CFG_DFG`.

Manual checklist нужен для тех областей, где автоматический анализ недостаточен. Некоторые вопросы нельзя надежно решить только инструментами: корректна ли экономическая модель, достаточно ли защищена роль администратора, безопасна ли логика обновления ргоху, не слишком ли широки emergency-права, корректно ли выбраны oracle-источники. Поэтому платформа формирует задачи ручного аудита, но не выдает их за подтвержденные уязвимости.

5. Примеры типичных атак

5.1. Reentrancy при выводе средств

Один из наиболее показательных сценариев выглядит следующим образом. Контракт хранит балансы пользователей и содержит функцию `withdraw`. Внутри функции сначала проверяется, что баланс пользователя достаточен, затем выполняется внешний вызов на `msg.sender`, а после этого баланс уменьшается. На первый взгляд код логичен: пользователь запросил вывод, контракт отправил деньги, затем обновил учет.

Проблема в том, что `msg.sender` может быть не обычным кошельком, а контрактом. Когда у него вызывается `receive` или `fallback`, он может снова вызвать `withdraw` до того, как исходный вызов дошел до строки уменьшения баланса. В результате один и тот же баланс используется несколько раз. Даже если современный Solidity откатит часть операции из-за underflow, это не делает шаблон безопасным. Сам факт внешнего вызова до изменения состояния является нарушением принципа checks-effects-interactions.

Безопасный вариант обычно строится иначе: сначала проверяется условие, затем обновляется внутреннее состояние, и только потом выполняется внешний вызов. Дополнительно может применяться reentrancy guard, который запрещает повторный вход в функцию до завершения предыдущего вызова.

5.2. Авторизация через `tx.origin`

Предположим, контракт-кошелек разрешает вывод средств только владельцу и проверяет условие `tx.origin == owner`. Злоумышленник может создать вредоносный контракт и убедить владельца вызвать его функцию, например через фальшивый интерфейс. Вредоносный контракт внутри своей функции вызывает кошелек жертвы. Для целевого кошелька `tx.origin` все еще будет адресом владельца, потому что именно владелец инициировал всю транзакцию. Проверка пройдет, хотя непосредственным вызывающим является вредоносный контракт.

Такая атака особенно опасна тем, что пользователь действительно подписывает транзакцию своим кошельком, но не всегда понимает, какие внутренние вызовы будут выполнены. Поэтому для контроля доступа в смарт-контрактах обычно используют `msg.sender`, а не `tx.origin`.

5.3. Слабая случайность

Контракт может использовать выражение вроде `uint256(keccak256(abi.encodePacked(block.timestamp, block.prevrandao, msg.sender))) % 100` для определения победителя. Это выглядит как случайное число, но на практике источники не являются надежными для финансово значимых сценариев. Некоторые параметры блока могут быть частично предсказуемы или находиться под влиянием валидатора. Пользователь может повторять попытки или оценивать вероятность результата заранее.

В играх, лотереях, распределении редких NFT или выборе победителей такой подход может привести к манипуляциям. Если результат имеет ценность, случайность должна строиться через более надежную схему, например commit-reveal или внешний проверяемый источник случайности.

5.4. Ошибки учета токенов и балансов

Многие DeFi-контракты поддерживают внутренний учет: кто сколько внес, кому сколько полагается, сколько токенов выпущено, сколько резервов хранится. Если контракт принимает эфир через `receive`, но не обновляет внутреннюю переменную баланса, возникает рассинхронизация. Если контракт списывает внутренний баланс после внешнего вызова, появляется reentrancy-риск. Если при делении происходит округление в пользу одной стороны, может появиться постепенный drain через много мелких операций.

Такие ошибки не всегда выглядят как очевидная “уязвимая строка”. Часто они проявляются только как нарушение инварианта. Например, сумма внутренних балансов не должна превышать фактический баланс контракта. Если Echidna находит последовательность вызовов, где это условие нарушается, значит в модели учета есть дефект.

5.5. Ошибки upgradeable-контрактов

В проху-архитектуре логика контракта может обновляться, но состояние хранится отдельно. Это удобно для поддержки продукта, но создает дополнительные риски. Если функция инициализации не защищена, атакующий может вызвать ее повторно и назначить себя владельцем. Если новая версия контракта меняет порядок переменных состояния, старые данные могут интерпретироваться неправильно. Если право обновления принадлежит одному ключу без задержки или мультиподписи, компрометация этого ключа превращается в полный захват протокола.

Автоматические инструменты могут заметить часть таких проблем, но окончательная проверка upgrade-логики обычно требует ручного анализа архитектуры.

5.6. DoS через gas и внешние вызовы

Контракт может хранить массив пользователей и выполнять цикл по всем адресам для распределения наград. Пока пользователей мало, функция работает. Но когда массив становится большим, вызов может перестать помещаться в лимит газа. В результате критическая функция блокируется. Похожая проблема возникает, если контракт в цикле отправляет эфир внешним адресам: один получатель может намеренно отклонять перевод и ломать всю операцию.

Такие ошибки особенно опасны в функциях выплат, ликвидаций, распределения наград и закрытия раундов. Безопаснее проектировать pull-модель, где каждый пользователь сам забирает свою выплату, а не контракт пытается отправить средства всем сразу.

6. Выбор блокчейн-платформы

Solidity Audit Platform ориентирована на Solidity и EVM-совместимые сети. Основными целевыми средами являются Ethereum, Polygon, BNB Chain, Arbitrum, Optimism, Base, Avalanche C-Chain и другие сети, совместимые с EVM.

Такой выбор обусловлен зрелостью экосистемы. Для Solidity уже существует развитый набор инструментов: компилятор `solc`, Foundry, Slither, Mythril, Echidna и множество библиотек. Кроме того, большая часть DeFi, NFT и protocol-логики исторически создавалась именно вокруг EVM. Это делает EVM-проекты хорошей точкой входа для платформы автоматизированного аудита.

Поддержка Solana, Move-based сетей, Cairo/Starknet или Hyperledger Fabric требует отдельной архитектуры анализа. Там отличаются языки, модель памяти, модель аккаунтов, среда исполнения и типовые классы уязвимостей. Поэтому расширение за пределы EVM возможно только после стабилизации Solidity-направления.

7. Консенсус и место платформы в инфраструктуре

Solidity Audit Platform не вводит собственный алгоритм консенсуса, потому что она не является блокчейн-сетью. Платформа работает вне сети и анализирует исходный код до деплоя. Поэтому выбор между Proof of Work, Proof of Stake или другими механизмами не относится к внутренней архитектуре проекта.

При этом безопасность конечного смарт-контракта зависит от сети, в которую он будет развернут. Ethereum использует Proof of Stake. L2-сети добавляют собственные особенности, связанные с sequencer, rollup-механикой, задержками вывода и мостами. Другие EVM-совместимые сети имеют свои компромиссы в области валидаторов, управления и финальности. Платформа может помочь найти ошибки в коде, но она не заменяет анализ сетевых, мостовых и governance-рисков.

На текущем этапе проекту не нужен собственный токен или отдельная сеть. Это усложнило бы юридическую и техническую модель, не добавляя ценности основной функции — анализу Solidity-кода.

8. Смарт-контракты и токеномика самого проекта

Solidity Audit Platform не требует собственных production smart contracts для базовой работы. Ее основная функция — принимать пользовательский код, запускать анализаторы и возвращать отчет. Это классическая backend/API/on-prem архитектура, а не блокчейн-протокол.

В будущем смарт-контракты могут использоваться для дополнительных сценариев. Например, можно публиковать hash отчета в сети, чтобы подтвердить, что конкретная версия кода была проверена. Также возможны marketplace-сценарии для связи проектов и аудиторов, escrow для оплаты аудита или reputation registry. Однако такие функции потребуют отдельного проектирования, аудита и юридической оценки.

Рекомендуемая стратегия на текущем этапе — не запускать собственный токен. Продукт решает прикладную задачу безопасности, и токен не является необходимым элементом. Монетизация может строиться через подписки, оплату API, enterprise-лицензии и on-prem внедрения. Это снижает регуляторные риски и позволяет сосредоточиться на качестве анализа.

9. Юридические и этические аспекты

Платформа обрабатывает исходный код, который может быть коммерческой тайной, pre-launch проектом или частью критичной финансовой логики. Поэтому главный принцип — минимизация рисков для пользователя. Код не должен передаваться

третьим лицам без согласия владельца. Загруженные проекты должны храниться изолированно. Для enterprise-сценариев важна возможность локального развертывания, при котором код вообще не покидает инфраструктуру заказчика.

Не менее важна безопасность исполнения. Анализаторы запускаются в Docker-контейнерах, проект монтируется в read-only режиме, сеть для analyzer-контейнеров отключается, а ресурсы ограничиваются по CPU, памяти и количеству процессов. Архивы должны распаковываться безопасно, чтобы исключить path traversal и другие атаки через структуру `.zip`.

С этической точки зрения платформа должна явно сообщать об ограничениях. Автоматический анализ не является гарантией безопасности. Отсутствие findings не означает отсутствие уязвимостей. Некоторые результаты могут быть ложными срабатываниями, а некоторые ошибки могут быть пропущены. Особенно это касается бизнес-логики, экономических атак, сложных oracle-сценариев, upgradeable-архитектуры и протокольных инвариантов.

Если платформа обнаруживает критическую уязвимость в приватном коде, результат принадлежит владельцу проекта. Публичное раскрытие без согласия недопустимо. Если в будущем появится marketplace или публичные отчеты, потребуется отдельная responsible disclosure policy.

С точки зрения регулирования текущий продукт не является биржей, кошельком, кастодианом или платежной системой. Он не хранит средства пользователей и не выполняет транзакции от их имени. Поэтому AML/KYC не является базовым требованием для локальной или self-hosted версии. Однако эти вопросы могут стать актуальными, если проект расширится до marketplace, escrow, токенизированной модели или финансовых сервисов вокруг аудита.

10. Дорожная карта

Текущая beta-версия доказывает, что единый backend pipeline может принимать `.sol` файлы и `.zip` проекты, запускать несколько анализаторов, сохранять findings и показывать пользователю технические логи. Уже реализованы загрузка проектов, асинхронные задачи, Basic Scanner, Slither, Foundry, Mythril, Echidna, CFG/DFG, reentrancy correlation, manual checklist и live status через WebSocket.

Следующий этап — доведение платформы до стабильного release-ready состояния. Для этого требуется закрепить зависимости, подготовить `.env.example`, стабилизировать Docker-образы анализаторов, улучшить обработку ошибок, добавить smoke-тесты для analyzer images, улучшить health/readiness endpoints и привести статусы инструментов к строгой модели. Особенно важно отделять реальные уязвимости от ошибок конфигурации и от информационных результатов.

После этого развитие должно сместиться в сторону качества отчета. Пользователю нужен не просто raw stdout от Mythril или Foundry, а понятная классификация: что является уязвимостью, что является failing test, что является пропущенной проверкой, а что является задачей ручного аудита. Для этого потребуется развивать parser и classifier слой, группировку findings, фильтры, поиск, карточки сводки и безопасное отображение больших логов.

Дальнейший этап — интеграция в рабочий процесс разработчиков. Платформа может получить CLI-клиент, GitHub Actions, GitLab CI, webhooks, SARIF export, JSON report export, механизм baseline/suppression и анализ только измененных файлов. Это позволит использовать ее не только как web-интерфейс, но и как часть CI/CD.

Enterprise-направление предполагает on-prem развертывание, управление сроками хранения данных, журналы доступа, квоты, рабочие пространства организаций и интеграцию с корпоративными средствами авторизации. Расширение за пределы EVM имеет смысл рассматривать только после того, как Solidity/EVM pipeline станет достаточно стабильным.

11. Бизнес-модель

Основная ценность проекта — сокращение времени предварительного аудита, улучшение обратной связи для разработчиков и снижение риска очевидных ошибок до обращения к профессиональным аудиторам. Поэтому монетизация может строиться без собственного токена.

Для индивидуальных разработчиков и небольших команд возможна подписочная модель с ограниченным количеством анализов в месяц. Для команд — тариф с историей проектов, private uploads, JSON-отчетами, CI-интеграцией и расширенным pipeline. Для enterprise-клиентов наиболее естественна on-prem лицензия, потому что многие компании не захотят загружать приватный код во внешний SaaS.

Дополнительно возможна модель API credits. Легкие проверки стоят дешевле, тяжелые анализаторы вроде Mythril и Echidna расходуют больше кредитов. Такая модель справедливо отражает вычислительную стоимость разных типов анализа.

Professional services могут включать помощь в проектировании инвариантов для Echidna, настройку CI/CD, разработку пользовательских правил, подготовку отчетов и сопровождение команд перед внешним аудитом. Marketplace для ручных аудиторов возможен в будущем, но не должен быть частью ранней версии из-за высокой юридической и организационной сложности.

12. Риски проекта

Главный технический риск — ложные срабатывания и пропуски. Автоматические инструменты не понимают всю бизнес-логику протокола, а значит не могут гарантировать полную безопасность. Этот риск снижается за счет комбинирования нескольких анализаторов, сохранения технических логов, ручного checklist и честного позиционирования платформы как pre-audit layer.

Второй риск — нестабильность toolchain. Solidity-проекты отличаются версиями компилятора, структурой Foundry-проектов, зависимостями, настройками `foundry.toml` и наличием внешних библиотек. Для снижения риска нужны стабильные Docker-образы, smoke-тесты, аккуратная обработка `.zip` архивов и понятные сообщения об ошибках.

Третий риск связан с безопасностью пользовательского кода. Загруженный проект потенциально может быть вредоносным или содержать чувствительные данные. Поэтому важны изоляция анализаторов, отключение сети, ограничения ресурсов, read-only mount и политика хранения.

Бизнес-риск состоит в том, что пользователи могут воспринимать автоматический отчет как гарантию безопасности. Этого нельзя допускать. В интерфейсе и документации нужно явно указывать, что платформа помогает находить проблемы, но не заменяет полноценный аудит и не является сертификатом безопасности.

13. Метрики успеха

Для оценки продукта важны не только количество запусков, но и качество анализа. На уровне продукта стоит отслеживать число загруженных проектов, долю успешно завершенных pipeline, среднюю длительность анализа, количество частичных сбоев, частоту повторного сканирования после исправлений и использование API.

На уровне качества безопасности важны подтвержденные findings, частота ложных срабатываний по каждому инструменту, самые частые классы уязвимостей, время от finding до исправления и доля проектов, где high/critical findings исчезают после повторной проверки.

На уровне бизнеса важны переходы из бесплатного использования в платное, стоимость вычислений, удержание команд, количество enterprise-пилотов, нагрузка на поддержку и востребованность on-prem установки.

14. Заключение

Solidity Audit Platform решает практическую задачу Web3-разработки: превращает разрозненные инструменты анализа смарт-контрактов в единый, воспроизводимый и

понятный процесс. Платформа принимает код, запускает несколько независимых анализаторов, сохраняет логи, нормализует findings и помогает разработчикам быстрее увидеть опасные участки.

Ключевая ценность проекта заключается не в обещании полной автоматической безопасности, а в создании надежного предварительного слоя анализа. Такой слой помогает находить reentrancy, опасные внешние вызовы, ошибки авторизации, слабую случайность, нарушения инвариантов и проблемы тестовых сценариев еще до публикации контракта в сети.

Оптимальная стратегия развития — сохранять фокус на Solidity/EVM, не запускать собственный токен и не усложнять продукт отдельной блокчейн-инфраструктурой. Вместо этого проект должен развиваться как надежная SaaS/API/on-prem платформа для анализа смарт-контрактов, с сильной изоляцией выполнения, понятной классификацией результатов и честным разделением автоматических findings, технических ошибок и задач ручного аудита.